A CORE THESIS

COC

Cognitive Orchestration for Codegen

Killing Vibe Coding: The 5 Layers of the Human-on-the-Loop Developer

AUTHOR Dr. Jack Hong, Singapore Management University

VERSION 1.1 | March 2026

SERIES Terrene Foundation White Paper

COC: A Core Thesis

Cognitive Orchestration for Codegen

Dr. Jack Hong, Singapore Management University · Version 1.1 | March 2026 · CC BY 4.0

Executive Summary

Vibe coding, the practice of describing what you want and letting AI build it, captured the software industry's imagination in 2025. For prototypes and weekend projects, it delivers. For production systems, it fails along three predictable fault lines: the AI forgets your instructions as context fills up, follows internet conventions instead of yours, and takes the shortest path to working code, which is never the secure path.

The problem is not the AI model. Models improve monthly. The problem is the complete absence of institutional knowledge surrounding the model.

This paper introduces Cognitive Orchestration for Codegen (COC), a five-layer architecture that provides AI coding assistants with the organizational context, guardrails, and operating procedures they need to function as disciplined engineering partners. COC applies the Human-on-the-Loop philosophy, established in the CARE framework, directly to software development: the developer's primary contribution is not writing code, but defining and maintaining the institutional context that makes AI-generated code trustworthy.

COC is the first domain application of Cognitive Orchestration (CO), the domain-agnostic base methodology for structuring human-AI collaboration. CO defines eight first principles and a five-layer architecture that apply across any domain where AI operates under human oversight. COC instantiates these principles for software development. The CO specification and thesis are published separately (see Hong, 2026c). The principles are not new. The systematic organization of those principles into a five-layer framework is.

Disclosure

I built the Kailash ecosystem. The code is released under the Apache 2.0 open-source license, meaning the source code is publicly available, and anyone is legally entitled to use it, modify it, and build commercial products with it, permanently and irrevocably.

The underlying agentic workflow orchestration technology is covered by two patent filings (PCT/SG2024/050503 and P251088SG); the Apache 2.0 license includes an automatic patent grant (Section 3) that gives every user a perpetual, royalty-free right to use the patented technology.

---

1. The Brilliant New Hire With Zero Onboarding

Think about what happens when you hire a brilliant new engineer. They may have graduated top of their class. They may know seventeen programming languages and have contributed to open-source projects used by millions.

On their first day, they are useless.

Not because they lack capability, but because they lack context. They do not know that the payment service has a three-second timeout that cannot be changed because of a vendor contract. They do not know that the marketing team's API pulls data at 2 AM and will break if the schema changes without notice. They do not know that the last person who used that particular database migration pattern caused a four-hour outage on a Friday evening.

Good organizations solve this with onboarding. They pair the new hire with a senior engineer. They provide documentation. They have code review processes that catch violations of internal standards. They have CI/CD pipelines that enforce conventions. They have a culture that transmits institutional knowledge from experienced practitioners to newcomers.

Now consider what the industry did with AI-assisted development. It took the most capable "new hire" in the history of software engineering, an AI that can generate code faster than any human, and gave it zero onboarding. No documentation of internal standards. No pairing with senior knowledge. No code review for convention compliance. No enforcement of organizational patterns.

The result is exactly what you would expect. The code works. It passes basic tests. And it silently violates your architectural standards in ways that will cost months to untangle.

The answer is not to abandon AI-assisted development. The answer is to build the onboarding.

---

2. The Vibe Coding Promise vs. Reality

The term “vibe coding” was coined by Andrej Karpathy in February 2025 to describe a style of programming where you “fully give in to the vibes, embrace exponentials, and forget that the code even exists” (Karpathy, 2025). The idea resonated because it contained a genuine insight: AI models had become good enough at code generation that the bottleneck was shifting from writing code to describing intent.

For prototypes and weekend projects, this works. For production systems serving real users with real data, it collapses along three predictable fault lines.

Fault Line 1: Amnesia. AI coding assistants operate within a context window, a fixed amount of text they can hold in working memory. Your carefully written architectural guidelines, your database conventions, your API naming standards, all of it competes for space with the code being generated. As sessions grow, the AI begins to forget. Not dramatically, but in the quiet way that matters most: it stops following your patterns and starts following its training data’s patterns instead. You tell it to use your ORM for all database operations. It does, for the first three files. By the fifth, it has silently switched to raw SQL. No error. No warning. Just drift.

Fault Line 2: Convention Drift. Every mature engineering organization has conventions, patterns that emerged from hard-won experience. Maybe you learned the hard way that database migrations need to be idempotent. Maybe your team discovered that a particular API pattern causes race conditions under load. The AI knows none of it. It has been trained on the open internet, Stack Overflow answers, GitHub repositories, blog posts. When your conventions conflict with internet conventions, the AI follows the internet.

Fault Line 3: Security Blindness. AI models optimize for the most direct path to a working solution. Security is almost never the most direct path. Environment variables are less direct than hardcoded strings. Parameterized queries are less direct than string concatenation. Proper authentication flows are less direct than disabling auth for development and forgetting to re-enable it. Research confirms that developers using AI assistants produce code with security vulnerabilities at rates comparable to or exceeding manually written code, and critically, are more likely to believe their code is secure when it is not (Perry et al., 2023).

The uncomfortable truth: these three failures share a common root cause, and it is not model capability. The models are remarkably capable. The failures are failures of context. The AI does not know your organization, your conventions, your security policies, or what you learned last quarter when a deployment went wrong. Vibe coding fails because it treats

AI as if it already has this institutional knowledge. It does not. And no amount of model improvement will give it this knowledge, because this knowledge is yours, specific to your organization, your domain, your history, and your hard-won lessons.

---

3. The Real Problem: Not Model Capability, but Institutional Amnesia

Raw model capability is becoming a commodity. In 2024, GPT-4 was the clear leader. By 2026, Claude, Gemini, and numerous open-source models have reached or exceeded its performance across most coding tasks. The gaps continue to narrow. The models will keep improving. Prices will keep falling.

If your competitive advantage depends on which model you use, you have no competitive advantage. Everyone has access to the same models.

The real problem, the problem that model improvement cannot solve, is institutional knowledge. Organizations know more than they can articulate in any training dataset. This tacit knowledge, how things are actually done, which patterns work and which ones burned you, what the documentation does not say, is precisely what the AI lacks. It is also precisely what makes an engineering team effective.

Vibe coding treats the model as the entire solution. COC treats the model as one component in a system where institutional knowledge is the differentiator. The value hierarchy inverts:

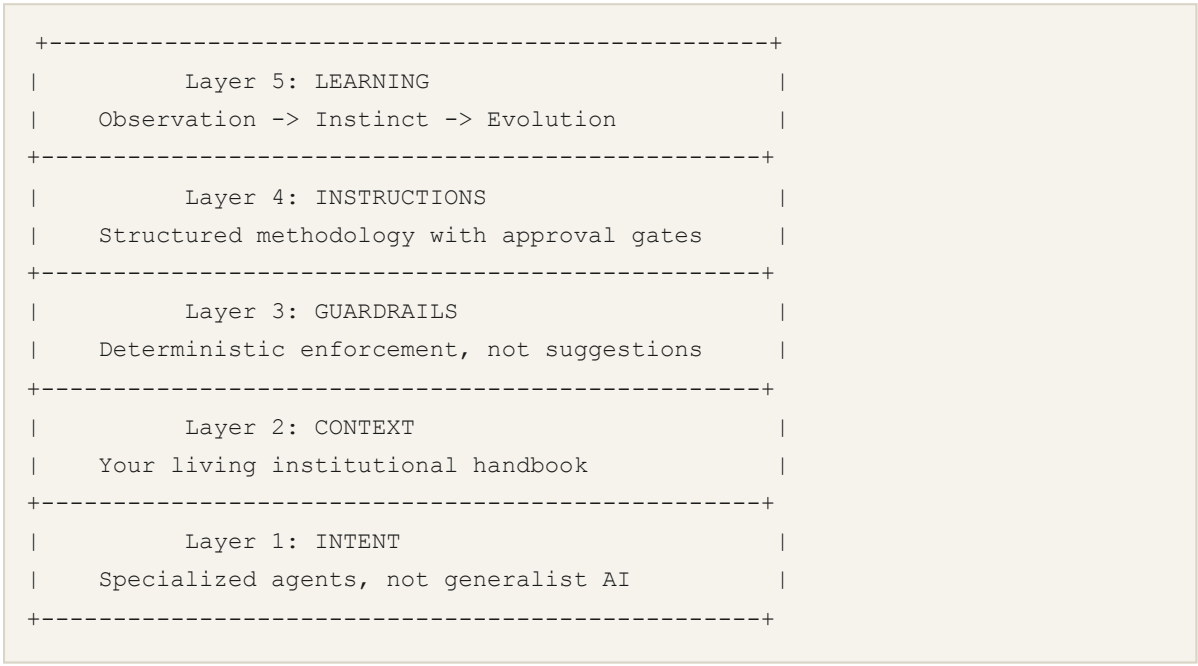
```
Vibe Coding Assumption:
  Better Model --> Better Code --> Competitive Advantage

COC Reality:
  Better Institutional Context --> Better AI Output --> Competitive Advantage
       (specific to you)          (using any model)    (defensible)
```

Your institutional knowledge, your frameworks, your conventions, your architectural decisions, your post-mortem lessons, your domain expertise, is specific to your organization. It cannot be downloaded. It cannot be replicated by a competitor switching to a better model. It is the accumulated wisdom of your team, and COC is an architecture for making it available to every AI interaction.

---

4. The Five-Layer Architecture

Cognitive Orchestration for Codegen (COC) is organized into five layers. Each addresses a specific failure mode of unstructured AI codegen. Together, they form the architecture for the Human-on-the-Loop Developer: a practitioner whose primary contribution is not writing code but defining and maintaining the institutional context that makes AI-generated code trustworthy.



This is not a theoretical framework. Each layer has a concrete implementation in the Kailash open-source ecosystem. The architecture descriptions that follow are drawn from the actual Kailash COC template repository (https://github.com/terrene-foundation/kailash-coc-claude-py).

Layer 1: Intent - The Role

What it solves: When you use a single generalist AI for everything, database design, API development, agent orchestration, frontend work, security review, you get generalist output. The AI applies its broadest training rather than deep domain-specific knowledge of your frameworks and patterns.

The principle: Stop asking one AI to be everything. Route tasks to specialized expert agents, each configured with deep knowledge of a specific domain.

In the Kailash ecosystem, this is implemented through 29 agent definitions organized across seven development phases: analysis, planning, implementation, testing, deployment, release, and frontend. Each agent has a defined specialty, specific tools it can use, a model tier

appropriate to its work (reasoning-heavy agents use Opus-class models for depth; speed-critical agents use Sonnet-class models for throughput), and curated domain knowledge.

Key specialists include:

- **deep-analyst**: Performs failure point analysis, 5-Why root cause investigation, and complexity scoring before any code is written.
- **security-reviewer**: Mandatory invocation before every commit. Not optional. Not "recommended." Mandatory.
- **Framework specialists** (dataflow, nexus, kaizen, mcp): Each carries deep knowledge of a specific framework's patterns, anti-patterns, and conventions.

The deeper principle mirrors how effective engineering organizations work. You do not ask the frontend developer to design the database schema, even if they technically could. You route database work to the database specialist. Layer 1 makes this routing explicit and systematic, so the AI brings domain-appropriate knowledge to every task.

Layer 2: Context - The Library

What it solves: AI training data is stale the moment training ends. Your internal frameworks, conventions, and architectural decisions are not in any training set. The AI defaults to internet conventions because it has no access to yours.

The principle: Replace the AI's stale training data with your living institutional handbook. Make your organizational knowledge the primary reference, not a secondary afterthought.

In the Kailash ecosystem, this is implemented through a structured knowledge hierarchy using progressive disclosure:

- **CLAUDE.md** (master directive): Loaded every session. Establishes ground rules – framework-first development, runtime execution patterns, testing policies, security requirements. This single file is the most important context document because it sets the baseline for every interaction.
- **SKILL.md index files** (10-50 lines): Quick-reference pattern guides. Enough to orient the AI on a topic without consuming context window space.
- **Topic files** (50-250 lines): Specific domain knowledge – particular frameworks, deployment patterns, testing strategies.
- **Full SDK documentation** (unlimited): Deep reference material pulled in only when needed for complex implementation work.

The implementation contains 25 skill directories with over 100 files, all following this progressive disclosure pattern. The AI loads the minimum context needed for the current task and pulls deeper reference only when required.

Two principles govern this layer.

The first is **Framework-First**: never code from scratch; always check frameworks first. The Kailash Core SDK provides 140+ production-ready nodes, which are pre-built building blocks for file I/O, HTTP operations, data transformation, validation, and dozens of other enterprise concerns. When the AI knows these exist (because the context tells it), it composes solutions from tested components rather than generating everything from scratch. This is the operational expression of the principle "composition over creation."

The second is **Single Source of Truth**: each piece of institutional knowledge lives in exactly one place, with no contradictions. When a convention changes, it changes in one file. The AI never encounters conflicting guidance.

This is what context engineering means, as distinct from prompt engineering. Prompt engineering optimizes a single interaction. Context engineering builds organizational knowledge that persists across every interaction, with every developer, across every session. The context is not a clever prompt. It is the institutional memory of your engineering team, made machine-readable.

Layer 3: Guardrails - The Supervisor

What it solves: Context documents tell the AI what to do. But AI models are probabilistic. They follow instructions most of the time, not all of the time. "Most of the time" is not acceptable for security-critical rules, architectural invariants, or infrastructure safety.

The principle: Deterministic enforcement, not probabilistic compliance. Not "the AI should do this" but "the system will not allow the AI to do that."

The implementation operates in two tiers.

Tier 1: Rules (8 files). These are soft enforcement – directives that the AI interprets and follows. They cover agents, testing policy, security practices, no-stubs policy, coding patterns, environment and model configuration, git workflow, and end-to-end testing. The AI reads these rules and incorporates them into its behavior. This works most of the time. Most of the time is not good enough for critical rules.

Tier 2: Hooks (8 scripts). These are hard enforcement - deterministic scripts that run on every tool call, outside the model's context window, independent of whether the AI "remembers" the rule. They intercept behavior and block violations before they reach the codebase.

The most important mechanism in the entire system is the **anti-amnesia hook**: `user-prompt-rules-reminder.js`. This hook fires on every single user message. Not every file write. Not every session start. Every message. It re-injects critical rules - environment summary, core behavioral rules, framework-first directives - fresh into the AI's context on every turn. It survives context window compression because it runs outside the model's memory. The AI's context window can fill up, old instructions can be evicted, and the anti-amnesia hook continues to enforce the rules because it does not depend on the model remembering them.

This is the single most important intervention in the architecture, because it directly addresses the most damaging failure mode: amnesia. The AI cannot drift past these boundaries no matter how long the session runs or how complex the task becomes.

Other hooks provide additional enforcement:

- **validate-workflow.js**: Catches wrong code patterns, hardcoded API keys (13 distinct detection patterns), hardcoded model names, stubs and TODOs left in production code, and mocking in integration tests.
- **validate-bash-command.js**: Blocks destructive commands (`rm -rf /` , fork bombs, `chmod 777 /`) and warns on risky commands (`curl | sh` , `git push` without security review).
- **session-start.js**: Establishes the environment baseline at the beginning of every session.

The architecture employs **defense in depth**: critical rules have five to eight independent enforcement layers. Consider the rule "never hardcode model names." This single rule is enforced by CLAUDE.md (loaded every session), by `rules/env-models.md` (soft rule), by `user-prompt-rules-reminder.js` (every user turn), by `session-start.js` (session start), and by `validate-workflow.js` (every file write). If any four of these mechanisms fail, the fifth catches the violation. This is not redundancy for its own sake. It is recognition that any single enforcement mechanism can fail, and that critical rules deserve multiple independent defenses.

This is the Trust Plane applied to development. In the CARE framework, the Trust Plane is where humans define boundaries that AI cannot override. Layer 3 is that same concept in the development context: human-defined rules, enforced deterministically, that the AI operates within. The AI has full autonomy to generate code, choose approaches, and solve problems – within the envelope. The envelope itself is not negotiable.

Layer 4: Instructions – The Operating Procedures

What it solves: Even with the right specialist, the right context, and the right guardrails, complex development tasks fail when there is no procedural discipline. The AI tackles the hardest part first when it should start with analysis. It writes code before confirming the approach. It claims completion without evidence.

The principle: Give the AI a structured methodology with approval gates between phases, evidence requirements for completion, and mandatory delegation rules.

The implementation provides a seven-phase development workflow – Analysis, Planning, Implementation, Testing, Deployment, Release, Final – with quality gates at four points: Planning, Implementation, Pre-commit, and Pre-push. The AI cannot skip phases. Each phase produces specific deliverables that the next phase consumes.

Twelve slash commands provide context-efficient invocation: `/sdk`, `/db`, `/api`, `/ai`, `/test`, `/validate`, `/design`, `/i-audit`, `/i-harden`, `/learn`, `/evolve`, `/checkpoint`. Each command activates the appropriate specialist agents, loads the relevant context, and establishes the procedural framework for the task.

Two elements of Layer 4 deserve particular attention.

Evidence-based completion: The AI cannot simply state "I implemented the feature." Every claim requires file-and-line proof. What file, what line, what test validates the behavior. This eliminates the confident hallucination, which is the AI's tendency to report success when it has actually introduced a subtle bug or missed a requirement.

Mandatory delegation rules: Code review after every file change. Security review before every commit. These are not suggestions. They are procedural requirements enforced by the workflow. The developer does not need to remember to ask for a security review. The system requires one.

The `/i-audit` command includes what the implementation calls an "AI slop test", which is a detection of telltale AI-generated aesthetic patterns that indicate the AI is producing generic output rather than project-specific code. This is a quality signal: when the output starts looking like AI wrote it for a blog post rather than for this specific project, something has drifted.

The approval gates between phases create natural points for human judgment. The developer does not monitor every keystroke of AI code generation. They review the analysis, approve the plan, verify the tests, and validate the evidence. This is human-on-the-loop applied to the development cycle: the human defines the procedure, the AI executes within it, and the human intervenes at structured checkpoints.

Layer 5: Learning – The Performance Review

What it solves: Most AI codegen workflows are stateless. Every session starts from zero. The AI learns nothing from yesterday's successes or failures. The team's institutional knowledge does not grow from AI-assisted development, it atrophies.

The principle: Observe what works, capture successful patterns, and evolve new capabilities over time. The institutional knowledge compounds with every session.

The implementation follows an Observation-Instinct-Evolution pipeline:

Observation (`observation-logger.js`): Logs tool usage, workflow patterns, errors, and fixes to structured JSONL files. Every interaction produces data about what patterns the team actually uses.

Instinct (`instinct-processor.js`): Analyzes accumulated observations for recurring patterns. It identifies:

- Workflow patterns: recurring node sequences (triggered at three or more occurrences)
- Error-fix pairs: errors followed by fixes within five minutes (triggered at two or more pairs)
- Framework selection patterns: which frameworks get chosen for which project types
- Each pattern receives a confidence score: frequency (40%) + success rate (30%) + recency (20%) + consistency (10%)

Evolution (`instinct-evolver.js`): When patterns reach confidence thresholds, the system suggests their evolution into formalized artifacts:

- Skills (confidence threshold 0.7 or above, five or more observations)
- Commands (confidence threshold 0.6 or above, three or more observations)
- Agents (confidence threshold 0.8 or above, ten or more observations)

The critical constraint: evolved artifacts are suggestions requiring human review before deployment. The system proposes. The human disposes. No pattern, regardless of its confidence score, becomes part of the institutional knowledge without a human approving it. The `checkpoint-manager.js` provides save, export, import, diff, and restore operations for the learning state, so teams can manage their evolving knowledge base with the same rigor they apply to code.

Over weeks and months, the system accumulates specialists for the organization's most common patterns. New team members inherit not just the codebase but the distilled judgment of everyone who has built with it. The AI becomes more effective not because the mod-

el improves (though it does), but because the institutional knowledge surrounding it deepens.

This is the most direct expression of the Human-on-the-Loop philosophy in the development context. The developer's primary value is not the code they write during any single session. It is the institutional knowledge they create and refine across all sessions. Layer 5 ensures that this knowledge is captured, structured, and made available to every future session.

---

5. The Quartet Integration: CARE, EATP, CO, PACT, and COC

COC is not a standalone framework. It inherits from a stack of interlocking standards published by the Terrene Foundation:

```
CARE (Philosophy)
  └── CO (Methodology)
        └── COC (Domain Application: Codegen)

EATP (Trust Protocol) ← mapped by all four
PACT (Organizational Architecture) ← operationalizes CARE governance
```

CARE provides the governance philosophy: the Dual Plane Model (Trust Plane + Execution Plane), the Mirror Thesis (AI reveals uniquely human value), Human-on-the-Loop governance, and the eight CARE principles. **CO** translates that philosophy into a domain-agnostic methodology: eight first principles, a five-layer architecture, three failure modes, and a conformance framework. **COC** instantiates CO for software development: specific agents, specific skills, specific hooks, specific workflows. **EATP** provides the cryptographic trust infrastructure that all four specifications reference: Genesis Records, Delegation Records, Constraint Envelopes, Capability Attestations, and Audit Anchors. **PACT** (Hong, 2026f) provides the organizational architecture that operationalizes CARE governance, defining how constrained organizations structure trust delegation, capability boundaries, and institutional knowledge management.

The mapping from CARE and EATP to COC is direct:

CARE / EATP Concept	COC Equivalent
Trust Plane (humans define boundaries)	Rules + CLAUDE.md
Execution Plane (AI at machine speed)	Agents + Skills
Genesis Record (initial trust anchor)	`session-start.js`
Trust Lineage Chain (traceability)	Mandatory review gates
Audit Anchors (proof of compliance)	Hook enforcement (exit code 2 blocks action)
Constraint Envelope (5 dimensions)	8 rule files + 8 hook scripts
Mirror Thesis (reveals human value)	Developer as CO Architect, not code writer
Human-on-the-Loop	Approval gates between phases

COC as Constrained Organization

The "Constrained Organization" (Hong, 2026e) describes a new organizational form where AI agents perform operational execution within cryptographically verifiable boundaries defined by human authority. Five constitutive properties characterize it:

1. **Architectural separation** of trust from execution
2. **Verifiable trust lineage** from human authority to agent action
3. **Structured institutional knowledge** encoded in a five-layer architecture
4. **Graduated autonomy** through named trust postures
5. **Compounding knowledge** that deepens with every interaction

COC is a concrete instantiation of this form in the software development domain. The CLAUDE.md and rule files are the Trust Plane. The agents and skills are the Execution Plane. The hooks enforce architectural separation. The mandatory review gates provide verifiable trust lineage. The five COC layers encode structured institutional knowledge. The phased workflow provides graduated autonomy. And Layer 5's observe-capture-evolve pipeline compounds knowledge across sessions.

The developer working within COC is operating a constrained organization of one: they define the envelope, AI executes within it, and the institutional knowledge compounds. This is the same pattern the Terrene Foundation uses at organizational scale, where 14+ specialized agents operate within constitutional constraints (77 clauses, 11 entrenched provisions) to manage the Foundation's knowledge base.

Relationship to CO

COC is the proof of concept for CO. It was built first, before CO was formalized, and the patterns that emerged in COC were generalized into the CO specification. The relationship is:

- **CO** defines the methodology: eight principles, five layers, three failure modes, conformance criteria
- **COC** demonstrates that methodology works in practice: specific implementations, measured outcomes, real deployment
- **CO for other domains** (Compliance, Finance, Operations, Healthcare, Education, Research) can reference COC as evidence that the five-layer architecture produces results

COC content folds into the CO paper for publication purposes. The CO paper (Hong, 2026c) cites COC as the reference implementation and draws on COC deployment experience for its empirical claims.

The Human-on-the-Loop Developer

In the CARE framework, the human-on-the-loop is not in the execution chain, not a bottleneck approving every action, but is an authority over the execution chain, defining boundaries, observing patterns, and refining the operating envelope based on what they learn.

In COC, the developer occupies exactly the same position. They are not writing every line of code (that is human-in-the-loop development, slow and defeating the purpose). They are not accepting whatever the AI produces without structure (that is vibe coding, fast and unreliable). They are on the loop: defining the intent routing (Layer 1), maintaining the institutional context (Layer 2), enforcing the guardrails (Layer 3), designing the operating procedures (Layer 4), and cultivating the learning systems that make every session more effective than the last (Layer 5).

The deeper parallel is this: CARE argued that the human's unique contribution in an AI-augmented enterprise is not doing the work but defining the boundaries and institutional knowledge that make the work valuable. COC makes the identical argument about software development. The developer's unique contribution is not the code generated in any single session. It is the institutional context they build and maintain across all sessions, context that turns a general-purpose AI into a domain-specific engineering partner. The CARE Mirror Thesis applies directly: when AI handles code generation, what remains visible is the developer's architectural judgment, domain expertise, quality standards, and accumulated

wisdom. These are the six human competencies (Ethical Judgment, Relationship Capital, Contextual Wisdom, Creative Synthesis, Emotional Intelligence, Cultural Navigation) applied to the engineering context.

Lisanne Bainbridge's "Ironies of Automation" (Bainbridge, 1983) observed that the more automated a system becomes, the more critical it is that human operators maintain deep understanding of the underlying process. COC is designed with this irony in mind. Every layer is an investment in the human's understanding: the intent layer forces the team to articulate their domain structure, the context layer forces them to document their institutional knowledge, the guardrails layer forces them to identify their non-negotiable rules, the instructions layer forces them to formalize their methodology, and the learning layer forces them to extract patterns from experience. The developer who builds COC layers becomes a deeper expert in their own domain, not a shallower one.

---

6. The Competitive Landscape: Convergence as Validation

COC does not operate in a vacuum. By mid-2026, every serious AI coding tool has converged on the same execution primitives.

Claude Code (Anthropic, February 2026) provides all seven execution primitives that COC describes: agents, skills, rules, hooks, commands, auto-memory, and settings – with hooks providing identical anti-amnesia functionality. Cursor's `.cursorrules` files serve a similar function to CLAUDE.md, providing project-specific context that the AI loads on every interaction. GitHub Copilot's custom instructions address amnesia by maintaining persistent directives. Aider's conventions system addresses convention drift by encoding project-specific patterns. Windsurf provides Memories and AGENTS.md for persistent context.

This independent convergence is not a threat to COC. It is validation. Multiple commercial tools, built by independent teams with different architectures, arrived at the same conclusion: raw model capability is insufficient, and what surrounds the model matters more than the model itself. The CO thesis calls this the "Institutional Knowledge Thesis." Claude Code calls it "context engineering." They are the same insight.

What Converged (Era 1: Execution Primitives)

Primitive	Claude Code	Cursor	COC (CO Layer)
Agent specialization	`.claude/agents/`	Agent mode	Layer 1
Context engineering	`CLAUDE.md` + skills	`.cursorrules`	Layer 2
Enforcement hooks	Pre/Post hooks	Custom rules	Layer 3
Tool permissions	allow/ask/deny	File permissions	Constraint envelopes (partial)
Session memory	Auto-memory	Context persistence	Layer 5 (partial)
Structured invocation	Slash commands	Composer mode	Layer 4 (partial)

At Layers 1-3, the convergence is near-complete. The mechanisms are commodity. Any team can replicate them with any tool that supports project-level configuration and pre-action scripts.

What Remains Unique (Era 2: Governance Methodology)

The divergence falls into two categories:

Structured workflows with quality gates (CO Layer 4): No shipping tool implements phase progression with enforced approval gates, evidence-based completion, mandatory delegation, and phase-skip prevention. Claude Code has commands (invocation). CO requires orchestration (state machines with enforcement). This is a MUST-level CO conformance requirement that no execution tool satisfies.

Institutional learning pipeline (CO Layer 5): No shipping tool implements the observe-capture-evolve pipeline with confidence thresholds, pattern formalization proposals, and human-gated promotion to institutional knowledge. Claude Code has auto-memory (unstructured free text). CO requires a learning architecture with structured observation, confidence scoring, and human-approved evolution.

Governance integration: No execution tool addresses the CARE Trust Plane, the EATP trust lineage chain, or multi-vendor governance. These are different problems at a different architectural layer. Execution tools answer “is this tool allowed?” (access control). CARE +

EATP answer “who authorized this action, within what constraints, and can we prove it to an auditor?” (trust governance).

COC’s Publication Status

Because of this convergence, COC is not being submitted as a standalone publication. The execution primitives (L1-L3) are now commodity – publishing them as novel would be dishonest. The governance methodology (L4-L5) and trust integration (EATP) are genuinely novel, but they belong in the CO paper, not in a codegen-specific paper.

COC remains the reference implementation for CO. The CO paper (Hong, 2026c) cites COC deployment experience as empirical evidence. Teams building CO for other domains (six domain applications are defined in the CO specification, including Compliance, Finance, Operations, Healthcare, Education, and Research) reference COC as proof that the five-layer architecture works in practice.

Any team can take individual elements from COC and apply them to their existing tools. The CLAUDE.md pattern works with any AI coding assistant that supports project-level configuration. The hook-based guardrails pattern works with any tool that supports pre-action scripts. The value is in the systematic combination, the governance integration, and the institutional discipline that the five-layer structure encourages.

---

7. What This Does Not Solve

Honesty requires acknowledging what COC cannot address.

Novel architecture decisions. COC excels at ensuring AI follows established patterns. It does not help when there is no established pattern, when the team confronts a genuinely novel architectural challenge that requires creative engineering judgment. Layer 2 can document the decision after it is made, but the decision itself remains a deeply human activity.

Distributed systems complexity. The guardrails in Layer 3 catch local violations, such as wrong patterns in individual files, dangerous commands, security anti-patterns. They do not catch emergent problems in distributed systems, such as race conditions that appear only under load, consistency failures across service boundaries, cascading timeout effects. These require system-level thinking that current AI codegen tools do not provide, regardless of the surrounding architecture.

Team dynamics and culture. COC structures the technical relationship between developer and AI. It says nothing about how to handle disagreements about architecture, how to mentor junior developers whose primary tool is AI, or how to maintain engineering culture when much of the code is machine-generated. These are organizational challenges that require organizational solutions.

Model-specific limitations. COC mitigates many model limitations, such as amnesia, convention drift, security blindness, but cannot eliminate them entirely. The AI will still occasionally produce incorrect code that passes all guardrails. The developer must still exercise judgment. The architecture reduces the frequency and severity of failures; it does not eliminate them.

Legacy codebases. When the codebase was not built with framework-first principles, when there are no consistent frameworks to compose with, the benefits of COC are reduced. Layer 3 (guardrails) and Layer 4 (instructions) still provide value, but the composition-over-creation advantage of Layer 2 requires frameworks to compose with.

---

8. From Commodity Models to Institutional Advantage

The strategic argument behind COC is worth stating plainly.

Every team in the world has access to the same AI models. The models will keep getting better and cheaper. If your development advantage depends on which model you use, that advantage evaporates with every model release.

What cannot be replicated is your institutional knowledge. Your COC layers encode what is specific to your organization:

- **Layer 1 (Intent)** encodes your organizational structure – which specialists matter, what domains you operate in.
- **Layer 2 (Context)** encodes your institutional knowledge – your conventions, frameworks, and hard-won lessons.
- **Layer 3 (Guardrails)** encodes your risk tolerance – what your organization considers safe and what it does not.
- **Layer 4 (Instructions)** encodes your process maturity – how your organization develops and validates software.
- **Layer 5 (Learning)** compounds all of the above – each successful project makes the next one better.

A team that has invested six months building COC layers has an advantage that no model upgrade can replicate. Their AI produces code that follows their architecture, respects their security policies, uses their frameworks, and embodies their engineering culture. A competitor using a better model but lacking this institutional context will produce faster code that does not fit their organization.

This is the development equivalent of "doing better things instead of doing things better." Vibe coding is doing things better by generating code faster. COC is doing better things by building the institutional infrastructure that makes every AI interaction produce code worth keeping.

References

- Bainbridge, L. (1983). "Ironies of Automation." *Automatica*, 19(6), 775-779.
- Hong, J. (2026). "Constraint Theater: Governance Without Wealth Effects." Submitted to *American Economic Review*. Theoretical foundation for this paper series.
- Hong, J. (2026a). "CARE: Collaborative Autonomous Reflective Enterprise — A Core Thesis." White Paper Series, Version 2.1. Terrene Foundation. https://github.com/terrene-foundation/publications/blob/main/CARE-Core-Thesis.pdf.
- Hong, J. (2026b). "Enterprise Agent Trust Protocol (EATP) — A Core Thesis." White Paper Series, Version 2.2. Terrene Foundation. https://github.com/terrene-foundation/publications/blob/main/EATP-Core-Thesis.pdf.
- Hong, J. (2026c). "Cognitive Orchestration (CO) — A Core Thesis." White Paper Series, Version 1.1. Terrene Foundation. https://github.com/terrene-foundation/publications/blob/main/CO-Core-Thesis.pdf.
- Hong, J. (2026d). "Cognitive Orchestration for Codegen (COC): A Core Thesis." White Paper Series, Version 1.1. Terrene Foundation. https://github.com/terrene-foundation/publications/blob/main/COC-Core-Thesis.pdf. (This paper.)
- Hong, J. (2026e). "The Constrained Organization: An Organizational Model for Enterprise AI Governance." White Paper Series, Version 1.0. Terrene Foundation. https://github.com/terrene-foundation/publications/blob/main/Constrained-Organization-Thesis.pdf.
- Hong, J. (2026f). "PACT: Principled Architecture for Constrained Trust – A Core Thesis." White Paper Series, Version 1.0. Terrene Foundation. https://github.com/terrene-foundation/publications/blob/main/PACT-Core-Thesis.pdf.

- Karpathy, A. (2025). “Vibe Coding.” Personal blog post, February 2025.
- Perry, N., Srivastava, M., Kumar, D., & Boneh, D. (2023). “Do Users Write More Insecure Code with AI Assistants?” *ACM CCS 2023*.
- Polanyi, M. (1966). *The Tacit Dimension*. University of Chicago Press.

The Kailash stack is open source under Apache 2.0. The COC setup repository containing the complete COC implementation is publicly available on GitHub.

A Note on How This Paper Was Produced

This paper was written using the process it describes.

The initial draft was generated by a team of AI agents:

- **research analysts** that explored the knowledge base and source repositories, wrote structured drafts, and cross-referenced claims against anchor documents.
- A separate **alignment-critic agent** red-teamed the output, identifying eight issues including factual errors, overstatements, and inconsistencies with source material.
- A **debate-expert agent** challenged the draft’s arguments and flagged unverifiable claims.

The author directed every iteration. Instructions included:

- remove claims that cannot be verified
- strip out references to structures that do not yet exist
- simplify the disclosure to plain facts
- explain technical terms for non-technical readers
- ensure the paper reads as a standards proposition rather than marketing

The author made direct edits to tone, phrasing, and substance throughout. Multiple rounds of revision followed, each narrowing the gap between what the agents produced and what the author judged to be honest and useful.

The AI agents drafted. The author defined the boundaries, evaluated the output, caught what the agents missed, and decided what stayed. That is human-on-the-loop in practice, not a theoretical model, but the process that produced this document.

The tools used were Claude Code (Anthropic) with specialized subagents for research, alignment checking, and adversarial review. The full conversation history, including every instruction, every correction, and every piece of content the author rejected, is retained by the author.

Version History

Version	Date	Changes
1.0	February 2026	Initial thesis: five-layer architecture for codegen, vibe coding critique, anti-amnesia patterns, institutional knowledge engineering, Trinity integration
1.1	March 2026	CO principle count updated (seven → eight). Acknowledged convergence with execution tools as validation. Terminology alignment with CO v1.1

This paper is Hong (2026d), derived from the theoretical foundation in Hong, J. (2026). "Constraint Theater: Governance Without Wealth Effects." COC is the reference implementation demonstrating that specification quality (the q parameter) can be structurally improved. See also: Hong, J. (2026a). "CARE: A Core Thesis" for governance philosophy. Hong, J. (2026b). "EATP: A Core Thesis" for trust verification. Hong, J. (2026c). "CO: A Core Thesis" for the base methodology. Hong, J. (2026e). "The Constrained Organization" for institutional design. Hong, J. (2026f). "PACT: A Core Thesis" for organizational architecture.